

ESTRUCTURA + MEMORIA + RAZONAMIENTO (QWEN)

Excelente. Lo que tienes en `MEMORIA.txt` es un motor de memoria y razonamiento en tiempo de ejecución (Python + ChromaDB + CoT + Fact-Checking + MCP). La estructura `.ai/` que ya definimos es el cerebro estático (prompts, reglas, contexto). La clave para potenciar tu proyecto es conectar ambos mundos: `.ai/` dicta *cómo debe usar* la memoria, y el motor Python es la memoria. Aquí tienes la integración práctica, lista para copiar y ejecutar.

1. Arquitectura Híbrida Recomendada

```
text
mi-proyecto/
├── .ai/
│   ├── system.md
│   ├── rules.md
│   ├── context.md
│   ├── agents.md
│   ├── tools.md
│   └── memory.md
├── y sync
├── memory_engine/
│   ├── memory.py, mcp_server.py...
├── .ai_memory/
├── opencode.json
└── src/

# 📖 Cerebro estático (prompt/config)
# + directivas de memoria, citas y CoT
# + hard constraints anti-alucinación
# + esquema de recuperación
# + rol "Memory Keeper" / "Auditor"
# 🛠️ NUEVO: herramientas MCP permitidas
# 🧠 NUEVO/ACTUALIZADO: políticas, schema

# ⚙️ Código de MEMORIA.txt (router.py,
# 📁 Base ChromaDB + exportaciones Git
# + config MCP
```

2. Contenido Actualizado para `.ai/` (Copiar y pegar)

- ♦ `.ai/tools.md` (NUEVO)

markdown

Herramientas MCP Permitidas

El agente DEBE usar estas herramientas expuestas por el motor de memoria (`memory_engine``). No ejecutar comandos externos no listados.

| Herramienta | Uso | Ejemplo de args |

|---|---|---|

| `architect\_search` | Recuperar contexto verificado | `{ "query": "cómo manejar rate limits", "type": "architecture", "limit": 3 }` |

| `architect\_save` | Guardar decisión/patrón | `{ "query": "...", "response": "...", "type": "decision", "why": "evitar timeout", "tags": ["api", "resilience"] }` |

| `architect\_verify` | Validar afirmación antes de responder | `{ "context": "[resultado de search]", "claim": "el límite es 100 req/min" }` |

| `architect\_export` | Exportar memoria para Git | `{}` |

⚠️ **Regla:** Si una respuesta contiene datos numéricos, fechas o afirmaciones técnicas, DEBES pasarla por `architect\_verify` antes de entregarla.

♦ [.ai/memory.md](#) (ACTUALIZADO)

markdown

Políticas de Memoria y Razonamiento

Este proyecto acumula conocimiento estructurado. El agente debe seguir este ciclo:

🔄 Ciclo de Vida

1. **Retrieve:** Llamar a `architect\_search` antes de responder consultas complejas.
2. **Reason:** Usar Chain of Thought para problemas >2 pasos.
3. **Verify:** Validar claims con `architect\_verify` o citas explícitas `[1]`, `[2]`.
4. **Save:** Loguear la interacción con `architect\_save` si genera un patrón reutilizable.
5. **Sync:** Exportar periódicamente (`architect\_export`) para versionado en Git.

📦 Schema de Guardado

Cada entrada debe contener:

- `type`: episodic | decision | architecture | bugfix | tutorial
- `why`: contexto de la decisión
- `tags`: 2-4 keywords técnicas
- `check\_conflicts`: true (obligatorio)

🛡️ Retención y Conflictos

- No guardar respuestas triviales o genéricas.
- Si `architect\_save` retorna `conflict\_detected`, alertar al usuario y proponer fusión.
- La memoria es por proyecto (`.ai\_memory/<proyecto>/`), pero permite cross-reference vía tags.

♦ `.ai/system.md` (AÑADIR AL FINAL)

markdown

🧠 Integración con Memoria y Razonamiento

- Antes de implementar o explicar algo complejo, consulta `architect\_search`.
- Si usas información externa o numérica, cita la fuente o pasa por `architect\_verify`.
- Al finalizar tareas que generen patrones reutilizables, ejecuta `architect\_save`.
- Mantén un tono técnico, preciso y verificable. La alucinación es inaceptable.

♦ [.ai/rules.md](#) (AÑADIR SECCIÓN)

markdown

```
## 🛡️ Anti-Alucinación y Verificación
- NUNCA afirmes datos técnicos sin soporte en `architect_search` o citas verificables.
- Si el contexto es insuficiente, responde: "No tengo evidencia verificable. ¿Deseas que busque en la web o guarde esta hipótesis como episódica?"
- Todo cambio arquitectónico debe registrar `why` y `learned` en memoria.
- Prohibido usar `any`, `TODO:` en producción, o hardcodear paths sin referencia en `context.md`.
```

🔌 3. Cómo conectar el motor Python con OpenCode/IDEs

El código de [MEMORIA.txt](#) ya expone un servidor MCP. Solo necesitas registrarlo en tu editor.

Paso 1: Ejecutar el servidor MCP

bash

```
cd mi-proyecto
```

```
pip install -r memory_engine/requirements.txt
```

```
python memory_engine/main.py --ui mcp
```

Verás: 🔌 Servidor MCP iniciado (stdio mode)

Paso 2: Configurar tu agente (OpenCode / Cursor / Claude Code)

En [opencode.json](#) (o [.cursorrules](#) / [claude_desktop_config.json](#)):
json

```
{
  "mcpServers": {
    "ai-memory-engine": {
      "command": "python",
      "args": ["memory_engine/main.py", "--ui", "mcp"]
    }
  }
}
```

✓ El agente ahora verá automáticamente: `architect_search`, `architect_save`, `architect_verify`, etc.

4. Flujo de Acumulación de Experiencia (Cómo "aprende")

Fase	Qué ocurre	Dónde se guarda
1. Ingesta	Tú o el agente suben docs/PRs/errores	ChromaDB (<code>.ai_memory/</code>) chunkeado + embeddings
2. Recuperación	<code>architect_search</code> trae top-k semántico + filtros	Memoria LTM/Episódica
3. Razonamiento	CoT estructurado (<code>Thought</code> → <code>Action</code> → <code>Observation</code>)	Prompt en memoria
4. Verificación	<code>architect_verify</code> cruza claim vs evidencia	Guardrails en memoria
5. Persistencia	<code>architect_save</code> loguea decisión + metadata	ChromaDB + tags + provenance
6. Sincronización	<code>architect_export</code> → JSON versionado en Git	<code>.ai_memory/export.json</code> (commiteable)

 Con el tiempo, el motor:

- Deduplica entradas por `signature` (hash del contenido)
 - Rerankéa por `hybrid_score > importance > recency`
 - Detecta conflictos semánticos con LLM-judge gratuito
 - Permite `import` en otros repos sin romper consistencia
-

✓ 5. Checklist de Implementación Rápida

- Crear carpeta `memory_engine/` y pegar el código de `MEMORIA.txt`
- Ejecutar `python memory_engine/main.py --ui mcp` y validar que expone herramientas
- Actualizar `.ai/tools.md`, `.ai/memory.md`, `.ai/system.md`, `.ai/rules.md` con los bloques anteriores
- Configurar `opencode.json` (o tu editor) para apuntar al servidor MCP
- Probar: `architect_save` → `architect_search` → `architect_verify` → `architect_export`
- Commitear `.ai_memory/export.json` periódicamente para backup/Git sync

💡 ¿Qué gano con esto?

Antes	Ahora
La IA "olvida" entre sesiones	Memoria persistente vectorial + episodios
Respuestas genéricas o inventadas	Citas, verificación automática y guardrails
Conocimiento disperso en chats	Base estructurada exportable, versionable y multi-proyecto
Sin razonamiento explícito	CoT + herramientas + ciclo de retroalimentación

¿Quieres que te genere:

1. 📦 Un `docker-compose.yml` que levante el MCP server + Chroma persistente?
2. 🛠️ Un script de `pytest` que valide el ciclo completo (save → search → verify → export)?
3. 🔄 Un adaptador para inyectar esto directamente en `opencode` sin MCP (modo `@file`)?